

Compendio: Arquitectura, Ingeniería Moderna y Fundamentos de Lenguajes

Generado por Gemini

24 de noviembre de 2025

Índice

1. Introducción	3
I Patrones de Diseño (GoF)	3
2. Patrones Creacionales	3
2.1. Builder (Constructor)	3
3. Patrones Estructurales	3
3.1. Decorator (Decorador)	3
3.2. Adapter (Adaptador)	4
3.3. Proxy	4
3.4. Composite (Compuesto)	4
3.5. Facade (Fachada)	4
4. Patrones de Comportamiento	5
4.1. Strategy (Estrategia)	5
4.2. State (Estado)	5
4.3. Template Method (Método Plantilla)	5
4.4. Visitor (Visitante)	5
II Resumen: Ingeniería de Software Moderna (David Farley)	6
5. Definición y Cambio de Paradigma	6
5.1. ¿Qué es la Ingeniería del Software?	6
5.2. El Método Científico en Software	6
6. Los Dos Pilares de la Ingeniería Moderna	6
6.1. 1. Expertos en Aprender	6
6.2. 2. Expertos en Manejar la Complejidad	7
7. Medición del Rendimiento (Modelo Accelerate)	7
7.1. Estabilidad (Calidad)	7
7.2. Flujo (Velocidad)	7
III Reflexión Crítica: El Abuso de los Patrones	8

8. Resumen: “Rules Are for Fools, Patterns Are for Cool Fools”	8
8.1. La Falacia de la Cantidad	8
8.2. El “Patrón para aplicar Patrones”	8
8.3. El Lado Oscuro	8
 IV Perspectiva Histórica: ¿Se ha simplificado la Computación?	 10
9. Resumen: Respuesta de Alan Kay en Quora	10
9.1. La Dilución Académica	10
9.2. El Giro Vocacional	10
9.3. La Verdadera Definición de “Computer Science”	10
 V Fundamentos de Lenguajes: Tipos y Method Lookup	 11
10.Sistemas de Tipos	11
10.1. Definición de Tipo	11
10.2. Declaración e Inferencia	11
10.3. Clasificación de Lenguajes	11
 11.Resolución de Métodos (Method Lookup)	 11
11.1. Definición	11
11.2. Implementación en Tipado Dinámico	12
11.3. Implementación en Tipado Estático	12

1. Introducción

Este documento consolida pilares fundamentales para el arquitecto de software: un catálogo de patrones de diseño (GoF), conceptos de Ingeniería de Software Moderna, una crítica a la sobreingeniería, una reflexión histórica sobre la naturaleza de la ciencia de la computación según Alan Kay, y fundamentos técnicos sobre sistemas de tipos y resolución de métodos.

Parte I

Patrones de Diseño (GoF)

2. Patrones Creacionales

2.1. Builder (Constructor)

Concepto: Separa la construcción de un objeto complejo de su representación, permitiendo crear diferentes representaciones con el mismo proceso.

Profundización Técnica: La potencia de este patrón radica en el uso de un *Director* que orquesta la construcción. Es ideal para crear objetos **inmutables**: configuras todo en el builder y el método `build()` devuelve una instancia cerrada.

- **Roles:**
 - *Builder (Interfaz)*: Define los pasos abstractos (ej: `buildPared`).
 - *ConcreteBuilder*: Implementa los pasos y mantiene el estado del objeto.
 - *Director*: Ejecuta los pasos en orden. No sabe qué construye, solo la “receta”.
 - *Product*: El objeto final.
- **El Problema:** El “Constructor Telescópico” (muchos parámetros, difícil de leer) y la necesidad de crear objetos complejos paso a paso.
- **Ejemplo Real:** Una hamburguesería. Tú (Cliente) pides al cajero. El cocinero (Director) sigue la receta. El ensamblador (Builder) pone los ingredientes.

3. Patrones Estructurales

3.1. Decorator (Decorador)

Concepto: Añade funcionalidades dinámicamente a un objeto envolviéndolo en un objeto de la misma clase base.

Profundización Técnica: Se basa en la **recursividad** y la **transparencia de interfaz**.

- **Mecánica:** El decorador tiene una instancia del componente base. Al llamar a `operacion()`, ejecuta su lógica extra y delega en `componente.operacion()`.
- **Diferencia con Herencia:** La herencia es estática (tiempo de compilación). El decorador es dinámico (tiempo de ejecución) y permite apilar múltiples comportamientos (ej. `new Gzip(new Buffered(new FileStream()))`).

3.2. Adapter (Adaptador)

Concepto: Permite que interfaces incompatibles colaboren.

Profundización Técnica: Existen dos variantes: de *Objetos* (composición) y de *Clase* (herencia múltiple).

- **Mecánica:** El adaptador “traduce” las llamadas. No solo cambia nombres de métodos, a menudo transforma datos (ej. convertir fechas de String a Date, o coordenadas polares a cartesianas) para que el sistema antiguo entienda al nuevo.
- **Uso común:** Generalmente es un patrón “a posteriori”, usado cuando no se puede refactorizar el código legado o de terceros.

3.3. Proxy

Concepto: Sustituto que controla el acceso a un objeto.

Profundización Técnica: El cliente no sabe que habla con un proxy. A diferencia del Decorator (que añade comportamiento), el Proxy controla el *acceso*.

- **Tipos de Proxy:**
 - *Virtual:* Carga diferida (Lazy Loading) de objetos pesados (imágenes, DB).
 - *Protección:* Verifica permisos de seguridad antes de delegar.
 - *Remoto:* Representa un objeto en otro servidor (serializa datos por red).

3.4. Composite (Compuesto)

Concepto: Compone objetos en estructuras de árbol para tratar a partes y todos de forma uniforme.

Profundización Técnica: La clave es la **uniformidad** recursiva.

- **Mecánica:** El contenedor recorre sus hijos llamando a la misma operación. Si el hijo es otro contenedor, repite el proceso; si es una hoja, ejecuta la acción.
- **Trade-off:** Gestión de hijos (**add**, **remove**). Si se ponen en la interfaz común, las hojas tienen métodos inservibles (rompe segregación de interfaz). Si se ponen solo en el contenedor, se pierde transparencia (requiere casting).

3.5. Facade (Fachada)

Concepto: Interfaz simplificada para una biblioteca compleja.

Profundización Técnica: No es solo un wrapper, es un **simplificador**.

- **Característica:** Una buena fachada gestiona dependencias y ciclos de vida complejos, pero **no bloquea** el acceso a las clases de bajo nivel si el usuario realmente necesita control granular.
- **Ejemplo:** Un motor de videojuegos. `Physics.Raycast()` es la fachada que oculta cálculos vectoriales y optimización espacial complejos.

4. Patrones de Comportamiento

4.1. Strategy (Estrategia)

Concepto: Define una familia de algoritmos intercambiables.

Profundización Técnica: Permite cambiar el “cerebro” de un objeto en tiempo de ejecución.

- **Diferencia con State:** En Strategy, el cliente **elige** activamente el algoritmo (ej. ordenar por fecha vs por nombre). En State, el cambio es interno y automático según el ciclo de vida.
- **Implementación moderna:** En lenguajes funcionales o con lambdas (Java, C#, Rust), las estrategias suelen pasarse como funciones simples, sin necesidad de crear clases completas.

4.2. State (Estado)

Concepto: Permite que un objeto altere su comportamiento cuando su estado interno cambia.

Profundización Técnica: Elimina las máquinas de estado gigantes basadas en **switch/case**.

- **Mecánica:** El objeto contexto delega en `estadoActual`.
- **Transiciones:** ¿Quién define el cambio? Puede ser el contexto o el propio estado (ej. el estado “Jugando” sabe que si la vida llega a 0, debe transicionar a “GameOver”). Esto último acopla los estados pero es más práctico.

4.3. Template Method (Método Plantilla)

Concepto: Define el esqueleto de un algoritmo, delegando pasos específicos a las subclases.

Profundización Técnica: Basado en el *Hollywood Principle* (“No nos llames, nosotros te llamamos”).

- **Estructura:**
 - *Método Plantilla (Final):* Orquesta la ejecución (`paso1(); paso2();`).
 - *Métodos Abstractos:* Obligatorios para la subclase.
 - *Hooks (Ganchos):* Métodos vacíos opcionales que la subclase puede sobrescribir para extender el proceso.

4.4. Visitor (Visitante)

Concepto: Añade operaciones a una estructura de objetos sin cambiar sus clases.

Profundización Técnica: Utiliza el **Double Dispatch** (Doble Despacho).

- **Mecánica:**
 1. El cliente llama a `elemento.accept(visitante)`.
 2. El elemento sabe quién es y llama a `visitante.visitElementoA(this)`.
 3. El visitante ahora tiene el tipo concreto y la instancia para operar.
- **Trade-off:** Excelente para agregar operaciones nuevas, terrible si la estructura de clases cambia (agregar un nuevo tipo de Elemento obliga a modificar todos los Visitantes).

Parte II

Resumen: Ingeniería de Software Moderna (David Farley)

Esta sección resume las ideas fundamentales presentadas en los primeros capítulos del libro *Modern Software Engineering*. El texto redefine la disciplina basándose en el empirismo y el método científico.

5. Definición y Cambio de Paradigma

5.1. ¿Qué es la Ingeniería del Software?

David Farley propone una definición operativa:

“La ingeniería del software es la aplicación de un método empírico y científico de encontrar maneras eficientes y económicas de solucionar problemas en software.”

A diferencia de otras disciplinas, el software no se trata de **producción** (que es esencialmente gratis, como copiar bits), sino que es un ejercicio puro de **diseño y exploración**. Por tanto, debemos aplicar técnicas de exploración (método científico) en lugar de técnicas de manufactura.

5.2. El Método Científico en Software

El desarrollo debe ser un proceso de descubrimiento. Para limitar el riesgo y progresar, se debe adoptar el ciclo científico:

1. **Caracterizar:** Observar el estado actual.
2. **Hipotetizar:** Crear una teoría.
3. **Predecir:** Qué sucederá según la hipótesis.
4. **Experimentar:** Verificar si se cumplió la predicción.

6. Los Dos Pilares de la Ingeniería Moderna

Para tener éxito, los ingenieros de software deben dominar dos competencias centrales:

6.1. 1. Expertos en Aprender

Dado que el desarrollo es exploración, la habilidad de aprender debe ser sostenible. Esto se logra mediante cinco comportamientos clave:

- **Iteración:** Avanzar en ciclos cortos para aprender rápido.
- **Feedback:** Obtener retroalimentación inmediata de los sistemas y usuarios.
- **Incrementalismo:** Construir el sistema paso a paso.
- **Experimentación:** Probar hipótesis mediante pruebas controladas.
- **Empirismo:** Basarse en datos reales y observación, no en suposiciones teóricas.

6.2. 2. Expertos en Manejar la Complejidad

Los sistemas modernos exceden la capacidad cognitiva humana. Para evitar el caos, se deben aplicar principios que controlen la complejidad:

- **Modularidad:** Dividir el sistema en partes manejables.
- **Cohesión:** Que cada módulo tenga un propósito claro y único.
- **Separación de responsabilidades:** Distintas partes del sistema manejan distintos problemas.
- **Abstracción:** Ocultar detalles innecesarios.
- **Bajo Acoplamiento:** Minimizar las dependencias entre módulos para facilitar cambios.

7. Medición del Rendimiento (Modelo Accelerate)

Contrario a la creencia popular, es posible medir el rendimiento de un equipo de software mediante métricas técnicas correlacionadas con el éxito del negocio. Se evalúan dos dimensiones:

7.1. Estabilidad (Calidad)

Mide la capacidad de mantener el sistema funcionando correctamente.

- **Tasa de fallos en cambios:** Porcentaje de despliegues que causan errores.
- **Tiempo de recuperación:** Tiempo necesario para recuperarse de un fallo.

7.2. Flujo (Velocidad)

Mide la capacidad de desplegar ideas y valor al cliente.

- **Tiempo de ejecución (Lead Time):** Tiempo desde la idea hasta el código funcionando en producción.
- **Frecuencia de despliegue:** Qué tan seguido se lleva software a producción.

Existe una correlación positiva: la velocidad y la calidad no son opuestas, sino que se habilitan mutuamente a través de la ingeniería.

Parte III

Reflexión Crítica: El Abuso de los Patrones

8. Resumen: “Rules Are for Fools, Patterns Are for Cool Fools”

Autor: Mahesh Dodani

Este artículo presenta una sátira narrativa a través del personaje ficticio “Pat Terna Buser” (un juego de palabras para “Pattern Abuser” o Abusador de Patrones), quien confiesa su adicción a aplicar patrones de diseño indiscriminadamente.

8.1. La Falacia de la Cantidad

El texto critica la tendencia de medir la calidad del software o la “reusabilidad” basándose únicamente en la **cantidad** de patrones implementados. El protagonista pasa de ser un programador pragmático a un “evangelista de patrones” que fuerza su uso en cada línea de código, obteniendo fama y fortuna consultora a costa de la mantenibilidad real del software.

8.2. El “Patrón para aplicar Patrones”

Para ilustrar el absurdo, el autor describe un algoritmo para forzar casi todos los patrones del libro *Gang of Four* en una clase simple de cuenta bancaria (**Account**):

- **State:** Se aplica a las variables de instancia (ej. **AccountState**).
- **Strategy:** Se aplica a cada método (ej. cálculo de intereses).
- **Chain of Responsibility:** Se usa para agrupar instancias y manejar cuentas sobregiradas dinámicamente.
- **Composite, Iterator, Visitor:** Para organizar las cuentas en estructuras de árbol y recorrerlas para operaciones como imprimir extractos.
- **Observer:** Para que los clientes y vistas observen cambios en el saldo.
- **Proxy:** Para interfaces distribuidas o servicios de cajeros.
- **Facade:** Para ocultar la complejidad masiva creada por los patrones anteriores.
- **Bridge:** Para separar la implementación del cálculo de intereses de su interfaz.

8.3. El Lado Oscuro

La conclusión de la sátira revela las consecuencias de este enfoque:

- **Complejidad Innecesaria:** El sistema se vuelve imposible de modificar para los desarrolladores originales.
- **Falsos Incentivos:** Se crea una necesidad artificial de consultoría para arreglar problemas creados por la propia sobre-ingeniería.
- **Preguntas Fundamentales:** El artículo cierra cuestionando cómo determinar cuándo un patrón es realmente aplicable y cómo definir si el software es objetivamente “mejor” tras su aplicación.

Lección Principal: Los patrones son herramientas para resolver problemas específicos de diseño, no metas en sí mismos. Su aplicación ciega conduce a sistemas frágiles y sobre-diseñados.

Parte IV

Perspectiva Histórica: ¿Se ha simplificado la Computación?

9. Resumen: Respuesta de Alan Kay en Quora

Alan Kay, pionero de la programación orientada a objetos, ofrece una visión crítica sobre la evolución de la “Ciencia de la Computación” (CS) desde los años 60 hasta la actualidad, argumentando que la disciplina ha sufrido una simplificación (“dumbing down”) desastrosa.

9.1. La Dilución Académica

- **De la Elite a la Masificación:** En los 60s, la “verdadera” ciencia de la computación se practicaba solo en pocas universidades de elite (MIT, CMU, Stanford). A partir de 1980, la disciplina se expandió rápidamente a miles de universidades.
- **Escasez de Talento:** Kay argumenta que no había suficientes profesores calificados para sostener la calidad educativa en miles de instituciones, lo que llevó a una caída inevitable en el nivel de enseñanza.

9.2. El Giro Vocacional

- **La Universidad como Negocio:** Las instituciones educativas se transformaron en negocios impulsados por la matrícula y el lucro, priorizando el “entrenamiento vocacional” (conseguir empleo) sobre la comprensión profunda.
- **El Síntoma Java:** Kay critica que universidades como Stanford adoptaran Java como lenguaje introductorio bajo presión del mercado, en lugar de enseñar las verdaderas “ciencias de la computación”.
- **Pérdida de Historia:** Los estudiantes actuales desconocen la historia del campo (ej. Douglas Engelbart) y carecen de curiosidad intelectual, tratando a la computación como una mera herramienta técnica en lugar de un campo de estudio profundo.

9.3. La Verdadera Definición de “Computer Science”

Para Kay y los pioneros de los 60s, la computación no era solo ingeniería o algoritmos, sino una ciencia en el sentido estricto:

1. **Ciencia de lo Artificial:** Siguiendo a Herb Simon, la computación estudia los artefactos creados por el hombre (como puentes o software) para entender sus fenómenos y crear mejores modelos.
2. **Ciencia de los Procesos:** Alan Perlis definió la CS como “la ciencia de los procesos; todos los procesos”.
3. **Sistemas sobre Algoritmos:** La computación trata sobre entender, inventar y construir **sistemas**. Dado que los sistemas complejos tienen dimensiones de tiempo y libertad enormes, deben ser “depurados” (debugged) más que “probados” matemáticamente.
4. **El Ciclo Virtuoso:** Entender cosas para permitir crear nuevas ingenierías que a su vez permitan entender cosas nuevas. El computador es un vehículo “meta” para modelar estas ideas.

Parte V

Fundamentos de Lenguajes: Tipos y Method Lookup

Esta sección consolida los conceptos teóricos y técnicos sobre sistemas de tipos y la resolución de métodos (dispatch), detallando su implementación en lenguajes estáticos y dinámicos.

10. Sistemas de Tipos

10.1. Definición de Tipo

En el contexto de la orientación a objetos y lenguajes de programación, un **Tipo** no es meramente una estructura de datos. Se define formalmente como:

[cite_sstart]“Un protocolo o conjunto de mensajes.”[cite : 1]

[cite_sstart]Desde una perspectiva **estructural**, un tipo se define por su nombre y por los mensajes (operaciones) que soporta [cite : 8].

10.2. Declaración e Inferencia

[cite_sstart]La asignación de tipos a las variables puede manejarse de tres formas principales [cite : 2] :

Explícito: El programador declara el tipo (ej. `int a` [cite: 3]). [cite_sstart]

Inferido: El compilador deduce el tipo (ej. `var b = 1;` [cite: 4]).

Combinado: Una mezcla de ambas estrategias.

10.3. Clasificación de Lenguajes

[cite_sstart]Los lenguajes se pueden clasificar según dos ejes : el momento en que se verifica el tipo (Estático/C dinámico) [cite : 9].

Sistema de Tipos	Estático (Compilado)	Dinámico (Ejecución)
Fuerte (Strong)	Java	Smalltalk, Ruby, Python [cite _s start]
Débil (Weak)	C++	VB6

Cuadro 1: Clasificación de lenguajes según su sistema de tipos [cite: 9].

11. Resolución de Métodos (Method Lookup)

11.1. Definición

El *Method Lookup* es el mecanismo fundamental mediante el cual el sistema encuentra el código ejecutable correspondiente a una llamada. [cite_sstart]Se define como la búsqueda del método a partir de los elementos [cite : 10, 11, 13] :

Un **Receptor** (el objeto que recibe el mensaje).

Un **Mensaje** (el nombre de la operación a ejecutar).

11.2. Implementación en Tipado Dinámico

En lenguajes dinámicos (como Smalltalk o Python), la búsqueda ocurre en tiempo de ejecución. [cite_{start}]*Debido a que esto puede ser costoso, existen varias técnicas de optimización* [cite : 14, 15, 16] :

- **DTS (Dispatch Table Search):** Búsqueda estándar en la tabla de métodos del objeto y sus ancestros.
- **GLC (Global Lookup Cache):** Una caché global que almacena los resultados recientes de búsquedas de métodos para evitar recorrer la jerarquía repetidamente.
- **IC (Inline Cache):** Guarda el resultado de la búsqueda directamente en el sitio de la llamada (en el código compilado/bytecode), asumiendo que el tipo del receptor no cambiará frecuentemente.
- **PIC (Polymorphic Inline Cache):** Una extensión del IC que puede almacenar múltiples resultados para el mismo sitio de llamada, optimizando para casos donde el receptor varía entre unos pocos tipos conocidos (polimorfismo).

11.3. Implementación en Tipado Estático

[cite_{start}]*En lenguajes estáticamente tipados (como C++), la resolución se optimiza utilizando una estructura como* [17].

Funcionamiento de la VTBL:

- **Creación:** Cada clase crea su propia tabla virtual [cite: 18]. [cite_{start}]
- **Dimensión:** El tamaño de la tabla es igual al tamaño de los selectores (métodos) de toda la jerarquía de la clase [cite: 19]. [cite_{start}]
- **Ordenamiento:** Se numeran los selectores desde arriba (desde la clase base) hacia abajo [cite: 20]. Esto garantiza que un método específico siempre tenga el mismo índice (offset) en la tabla, independientemente de la subclase. [cite_{start}]
- **Contenido:** En cada entrada de la VTBL se coloca el puntero a la dirección de memoria del código del método correspondiente [cite: 22].

Esto permite que la llamada a un método virtual sea una operación de indexación directa ($O(1)$), mucho más rápida que la búsqueda dinámica por nombre.